Weather Analytics
dbt + MotherDuck + Streamlit

A complete, plain-English walkthrough of every part of the project: the data pipeline, the dbt transformation layers (staging, intermediate, marts), the analytics logic, and the interactive dashboard.

Project	dbt-weather-analytics
Goal	Find the ideal European city for a holiday, from live weather data
Data source	Open-Meteo public API (weather, forecast, air quality)
Warehouse	MotherDuck cloud database (open_meteo_europe_sa)
Transformations	dbt (staging → intermediate → marts)
Dashboard	Streamlit web app (6 interactive pages)
Group members	Andrea Alarcón Valles, Tina Jannasch, Ricardo Liévano Pedroza, Luka Tcheishvili
Live dashboard	dbt-weather-analytics-y8gbhkpnqc27szlu9znbza.streamlit.app
Document generated	01 July 2026

This document is written for a non-technical reader. Every technical term is either explained where it first appears or defined in the Glossary (Section 18).

Contents

1. Introduction — what this project is

This project answers one practical question: **“Which European city has the best weather for a holiday right now, and over the next week?”** To do that, it pulls real weather data from a free public service, cleans and reshapes that data through several well-defined stages, calculates a single easy-to-read “visit score” for every city, and presents everything in a polished, interactive web dashboard.

The project is a classic example of the modern **analytics engineering** workflow. Rather than writing one giant, tangled program, the work is split into small, testable steps that each do one job. This makes the whole system easier to understand, to trust, and to change — which is exactly why companies build their data platforms this way.

The three pillars of the project

- **The pipeline (Python):** small scripts that *extract* data from the Open-Meteo API and *load* it into the cloud database.
- **The transformations (dbt + SQL):** the heart of the project. Raw data is progressively refined through three layers — **staging**, **intermediate**, and **marts** — until it is clean, enriched, and ready for analysis.
- **The dashboard (Streamlit):** a web application with six pages that lets a user explore the results visually — maps, charts, rankings and city deep-dives.

Who this document is for

This is a guided tour of the codebase for someone who does *not* need to be a programmer. For each file we explain, in everyday language, **what it does** and **why it matters**, and we show the most important piece of the actual code with an explanation of what that code achieves. The dbt transformation models (staging, intermediate, marts) are shown in full, because they are the analytical core of the project.

2. The big picture — how the data flows

Data enters on one side as raw numbers from the internet and comes out the other side as a friendly dashboard. In between, it passes through a series of stages, each one making the data a little cleaner and more useful. This staged approach is often called a **medallion architecture** (raw → refined → analytics-ready).

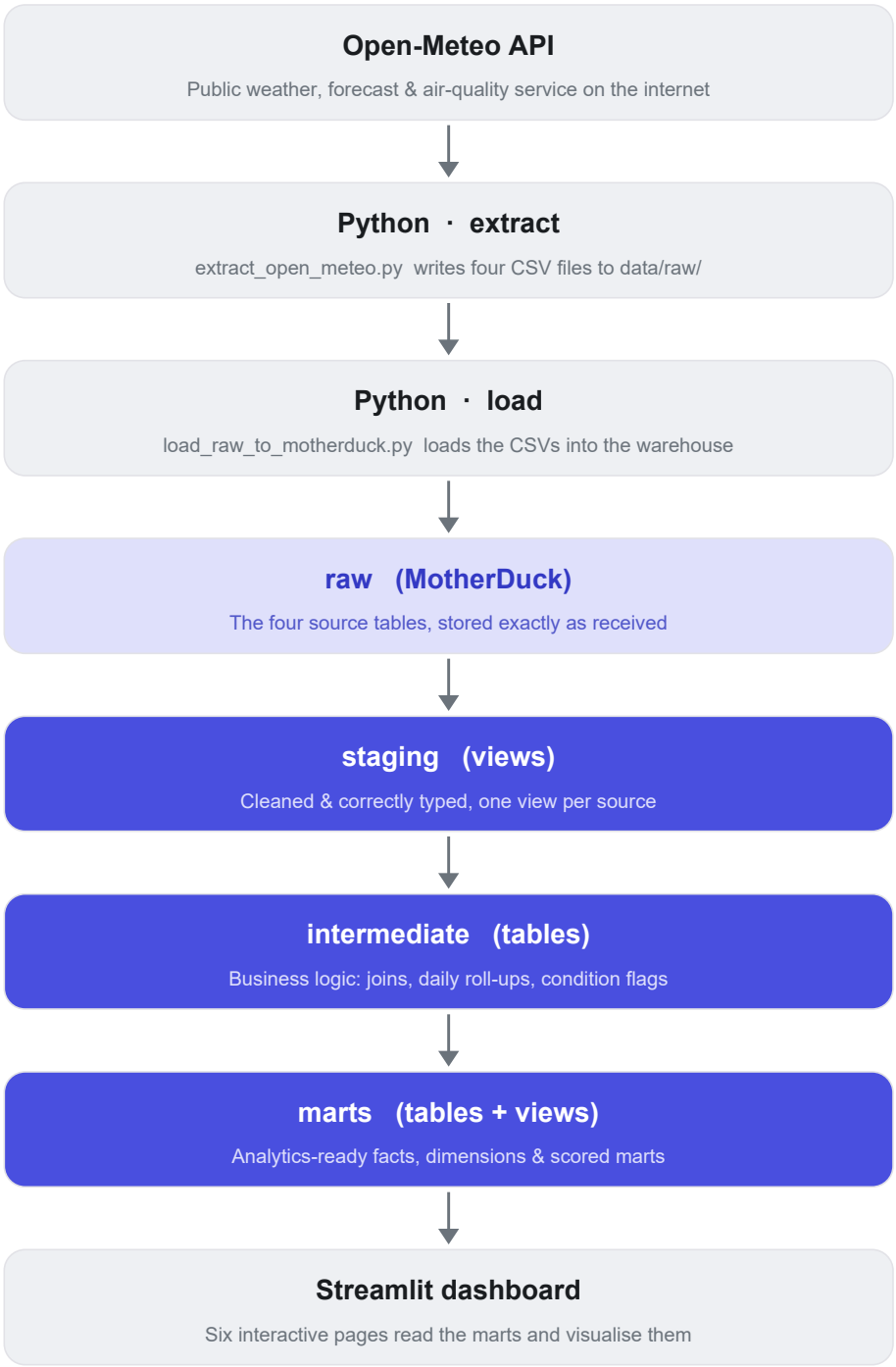

Figure 1 — End-to-end data flow. Each box hands cleaner data to the one below it.

Why split it into layers?

Each layer has a single, clear responsibility, so if something looks wrong you know exactly where to look. It also means the same clean building blocks can be reused by many different final outputs without repeating work.

- **raw** — an untouched copy of what the API sent. Never edited, so we always have the original.
- **staging** — light cleaning only: rename, fix data types, remove duplicates. One staging model per raw table.
- **intermediate** — the “thinking” layer: it joins tables together, summarises hourly data into daily data, and works out plain-English flags such as “is this a rainy day?”.

- **marts** — the finished products the dashboard actually uses: fact tables, a location dimension, and ready-scored recommendation marts.

3. Repository structure

Below is the full folder layout. Grey folders are standard dbt scaffolding; the important work lives in **models/** (the SQL transformations), **scripts/** (the Python pipeline) and **dashboard/** (the web app).

```
dbt-weather-analytics/
|
|-- dbt_project.yml # Main dbt configuration + tunable thresholds (vars)
|-- profiles.yml # How dbt connects to MotherDuck
|-- packages.yml # External dbt packages (dbt_utils)
|-- requirements.txt # Python libraries needed to run everything
|-- pyproject.toml # Alternative Python dependency definition
|-- README.md # Human-readable project guide
|-- AGENT.md # Design-system rules for the dashboard
|-- .env.example # Template for secret credentials
|
|-- scripts/ # The extraction + load pipeline (Python)
| |-- extract_open_meteo.py # Pull data from the Open-Meteo API -> CSV
| |-- load_raw_to_motherduck.py # Load the CSVs into the raw schema
| \-- bootstrap_motherduck.py # Create the database, schemas & raw tables
|
|-- sql/
| \-- init_motherduck.sql # SQL that creates the database objects
|
|-- data/raw/ # Sample extracted CSV data (committed for demos)
| |-- raw_locations.csv
| |-- raw_weather_daily.csv
| |-- raw_forecast_daily.csv
| \-- raw_air_quality_hourly.csv
|
|-- models/ # THE dbt TRANSFORMATIONS (the analytical core)
| |-- staging/ # Layer 1: cleaned, typed views
| | |-- _sources.yml # Declares the raw source tables
| | |-- _staging.yml # Descriptions + data tests
| | |-- stg_locations.sql
| | |-- stg_weather_daily.sql
| | |-- stg_forecast_daily.sql
| | \-- stg_air_quality_hourly.sql
| |-- intermediate/ # Layer 2: business logic & flags
| | |-- _intermediate.yml
| | |-- int_city_day_weather.sql
| | |-- int_air_quality_daily.sql
| | |-- int_weather_flags.sql
| | \-- int_forecast_daily_enriched.sql
| \-- marts/ # Layer 3: analytics-ready outputs
| |-- _marts.yml
| |-- dim_location.sql
| |-- fct_city_weather_day.sql
| |-- fct_forecast_accuracy.sql
| |-- mart_latest_conditions.sql
| \-- mart_forecast_upcoming.sql
|
|-- macros/
| \-- generate_schema_name.sql # Routes each layer to its own schema
|
|-- tests/ # Custom (singular) data-quality tests
| |-- assert_city_weather_temp_in_range.sql
| \-- assert_forecast_horizon_non_negative.sql
|
|-- analyses/sanbox.sql # Ad-hoc scratch query (not built into the DB)
|
|-- dashboard/ # THE STREAMLIT WEB APP
| |-- app.py # Entry point: header, theme, tabs
| |-- utils/
| | |-- db.py # Read-only, cached queries to the marts
| | \-- theme.py # All styling + reusable UI components
| \-- pages/ # One file per dashboard tab
| |-- overview.py
```

```
| |-- map_view.py
| |-- forecast.py
| |-- comparison.py
| |-- recommendations.py
| \-- city_detail.py
|
|-- .github/workflows/ # Automation (CI/CD)
| |-- open-meteo-raw-load.yml # Daily: extract -> load -> dbt build
| \-- motherduck-smoke-test.yml # Quick connection check
|
\-- .streamlit/config.toml # Streamlit brand colour
```

Complete file index

Every file in the repository, its one-line purpose, and where it is covered in this document.

File	What it is	Sec.
dbt_project.yml	Master dbt config; declares layers, materializations and threshold vars	5
profiles.yml	Connection details from dbt to MotherDuck (no secrets in code)	5
packages.yml	Third-party dbt package dependency (dbt_utils)	5
macros/generate_schema_name.sql	Puts each layer in its own named schema	5
sql/init_motherduck.sql	Creates the database, four schemas and raw tables	5
models/staging/_sources.yml	Declares the four raw source tables to dbt	6
models/staging/_staging.yml	Descriptions + tests for the staging models	7
models/staging/stg_locations.sql	Clean list of cities (one row per city)	7
models/staging/stg_weather_daily.sql	Clean observed daily weather	7
models/staging/stg_forecast_daily.sql	Clean daily forecasts (latest snapshot)	7
models/staging/stg_air_quality_hourly.sql	Clean hourly air-quality readings	7
models/intermediate/int_city_day_weather.sql	Weather joined to city attributes	8
models/intermediate/int_air_quality_daily.sql	Hourly air quality rolled up to daily	8
models/intermediate/int_weather_flags.sql	Boolean weather condition flags	8
models/intermediate/int_forecast_daily_enriched.sql	Forecasts + horizon (days ahead)	8
models/marts/dim_location.sql	The location dimension (one row per city)	9
models/marts/fct_city_weather_day.sql	Full observed-weather fact table	9
models/marts/fct_forecast_accuracy.sql	Forecast vs. actual, with errors	9
models/marts/mart_latest_conditions.sql	Latest conditions + 7-day visit score	9
models/marts/mart_forecast_upcoming.sql	Upcoming forecast + score components	9
tests/assert_city_weather_temp_in_range.sql	Test: temperatures must be plausible	12
tests/assert_forecast_horizon_non_negative.sql	Test: forecast horizon cannot be negative	12

File	What it is	Sec.
scripts/extract_open_meteo.py	Downloads data from the Open-Meteo API	13
scripts/load_raw_to_motherduck.py	Loads the CSVs into MotherDuck	13
scripts/bootstrap_motherduck.py	Sets up the database objects safely	13
dashboard/app.py	Dashboard entry point (header, theme, tabs)	14
dashboard/utils/db.py	Cached, read-only queries to the marts	14
dashboard/utils/theme.py	All CSS styling + reusable UI components	14
dashboard/pages/overview.py	Overview tab (KPI cards + top pick)	14
dashboard/pages/map_view.py	Interactive map of cities	14
dashboard/pages/forecast.py	Forecast & trends charts	14
dashboard/pages/comparison.py	Side-by-side city comparison	14
dashboard/pages/recommendations.py	Visit-score leaderboard + activity presets	14
dashboard/pages/city_detail.py	Per-city deep dive	14
.github/workflows/open-meteo-raw-load.yml	Daily automated pipeline run	15
.github/workflows/motherduck-smoke-test.yml	Lightweight connectivity check	15
.env.example	Template for the secret token file	16
.streamlit/config.toml	Sets the app's cobalt brand colour	16
requirements.txt / pyproject.toml	Python dependency lists	16
README.md / AGENT.md	Project guide / design-system rulebook	16
analyses/sanbox.sql	Throwaway scratch query	16

4. The technology stack in plain English

A few named tools appear throughout the project. Here is what each one is and the job it does here.

Tool	What it is	Its job in this project
Open-Meteo	A free public weather service on the internet (an API)	Provides the raw weather, forecast and air-quality numbers
Python	A general-purpose programming language	Runs the small scripts that download and load the data
MotherDuck	A cloud data warehouse (a hosted version of DuckDB)	Stores all the data and runs the SQL transformations
DuckDB	A fast analytics database engine	The engine MotherDuck is built on; also used locally
dbt	“data build tool” — organises SQL into tested, dependency-aware steps	Turns raw data into clean, analytics-ready tables
SQL	The standard language for querying databases	The language every dbt model is written in
Streamlit	A Python framework for building web apps quickly	Powers the interactive dashboard
Plotly	A charting library	Draws the line charts, radar charts and bar charts
Folium	A mapping library (Leaflet maps)	Draws the interactive city map
GitHub Actions	An automation service built into GitHub	Runs the whole pipeline automatically every day

5. The dbt project configuration

Before looking at the transformations themselves, it helps to see how the dbt project is wired up. These configuration files tell dbt where things go, how to connect, and which settings to use.

dbt_project.yml — the master settings file

This is the control panel for the whole dbt project. Two parts matter most. First, the **models** block says how each layer should be built: staging models become **views** (lightweight, always up-to-date virtual tables) while intermediate and marts become real **tables** (stored on disk for speed). Second, the **vars** block is a single place to tune the thresholds that decide what counts as a “rainy”, “hot” or “comfortable” day.

```
name: dbt_weather_analytics
version: "1.0.0"
config-version: 2

profile: dbt_weather_analytics

model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

clean-targets:
  - target
  - dbt_packages

models:
  dbt_weather_analytics:
    staging:
      +schema: staging
      +materialized: view
    intermediate:
      +schema: intermediate
      +materialized: table
    marts:
      +schema: marts
      +materialized: table

vars:
  # Intermediate-layer flag thresholds (single source of truth; tune here).
  rainy_precip_mm: 1 # precipitation_sum >= this -> rainy day
  hot_temp_c: 30 # temperature_2m_max >= this -> hot day
  windy_kmh: 38 # wind_speed_10m_max >= this -> windy day
  poor_aqi: 60 # max_european_aqi >= this -> poor AQI day
  comfortable_temp_min_c: 18 # mean temp lower bound for a comfortable day
  comfortable_temp_max_c: 26 # mean temp upper bound for a comfortable day
  freezing_temp_c: 0 # temperature_2m_min <= this -> freezing day
  complete_aqi_hours: 24 # hours_observed >= this -> complete AQI day
```

Why this matters: keeping the thresholds in one `vars` block means a professor or teammate can change the definition of a “hot day” (currently 30°C) in one line, and every model and dashboard number updates consistently. No hunting through the code.

profiles.yml — the database connection

This tells dbt how to reach the warehouse. Notice there is no password written here: the token is pulled from an environment variable (`env_var`) so secrets never end up in the code. The connection type is **duckdb**, and the `md:` prefix in the path tells DuckDB to connect to MotherDuck in the cloud.

```
dbt_weather_analytics:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: "md:{{ env_var('MOTHERDUCK_DATABASE', 'open_meteo_europe_sa') }}?motherduck_token={{ env_var('MOTHERDUCK_TOKEN')
}}"
      schema: marts
      threads: 4
```

macros/generate_schema_name.sql — tidy schema routing

A **macro** is a reusable snippet of logic. By default dbt would create oddly-named schemas like `marts_staging`. This macro overrides that behaviour so each layer lands in a clean schema named exactly `staging`, `intermediate` or `marts`.

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}
{%- endmacro %}
```

packages.yml — reusing trusted code

This pulls in **dbt_utils**, a popular free package of helper tests and macros. The project uses its `unique_combination_of_columns` test to prove that model grains (for example, “one row per city per day”) are never accidentally duplicated.

```
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.1.0", "<2.0.0"]
```

sql/init_motherduck.sql — creating the database objects

This is a one-time setup script. It safely creates the database, the four schemas, and the four empty raw tables that the pipeline will fill. Every statement uses `IF NOT EXISTS`, so running it again does no harm.

```
CREATE DATABASE IF NOT EXISTS open_meteo_europe_sa;
USE open_meteo_europe_sa;

CREATE SCHEMA IF NOT EXISTS raw;
CREATE SCHEMA IF NOT EXISTS staging;
CREATE SCHEMA IF NOT EXISTS intermediate;
CREATE SCHEMA IF NOT EXISTS marts;

CREATE TABLE IF NOT EXISTS raw.raw_locations (
    extracted_at TIMESTAMPTZ,
    location_id BIGINT,
    city_name VARCHAR,
    country VARCHAR,
    country_code VARCHAR,
    admin1 VARCHAR,
    latitude DOUBLE,
    longitude DOUBLE,
    timezone VARCHAR,
    elevation DOUBLE,
    population BIGINT
);

-- ... raw_weather_daily, raw_forecast_daily and
-- raw_air_quality_hourly tables follow the same pattern ...
```

6. Sources — the raw layer

Before dbt can transform data, it needs to know where the raw data lives. The `_sources.yml` file **declares** the four raw tables to dbt and documents every column. This gives two big benefits: dbt can trace exactly where each downstream table's data came from (its “lineage”), and every model refers to raw data through a single named reference rather than hard-coded table names.

Raw source table	One row per...	Holds
raw_locations	City	City name, country, coordinates, timezone, elevation, population
raw_weather_daily	City & day	Observed daily temperature, precipitation, rain, snow, wind
raw_forecast_daily	City, day & extraction run	Forecasted daily weather (multiple snapshots kept)
raw_air_quality_hourly	City & hour	PM2.5, PM10, CO, NO2, ozone and the European AQI

Here is the top of the sources declaration. The database name itself comes from an environment variable, with a sensible default, so the same code works locally and in the cloud.

```
version: 2

sources:
  - name: open_meteo
    database: "{{ env_var('MOTHERDUCK_DATABASE', 'open_meteo_europe_sa') }}"
    schema: raw
    description: Raw Open-Meteo data loaded into MotherDuck.
    tables:
      - name: raw_locations
        description: One row per selected city from the Open-Meteo Geocoding API.
        columns:
          - name: extracted_at
            description: UTC timestamp when the extraction ran.
          - name: location_id
            description: Open-Meteo geocoding location identifier.
          - name: city_name
            description: City name returned by the geocoding result.
          - name: country
            description: Country name.
          - name: country_code
            description: ISO-style country code from Open-Meteo.
          - name: admin1
            description: First-level administrative area.
          - name: latitude
            description: City latitude.
          - name: longitude
            description: City longitude.
          - name: timezone
            description: Local timezone returned by Open-Meteo.
          - name: elevation
            description: City elevation in meters.
          - name: population
            description: Population returned by the geocoding API, when available.
```

Excerpt: models/staging/_sources.yml (lines 1–34). The other three tables are declared the same way.

7. Staging layer — cleaning the raw data

The staging models are the first transformation. Each one takes exactly one raw table and produces a clean, correctly-typed version of it — nothing more. They are built as **views**, meaning they are always live and take up no storage. Every staging model follows the same two-step recipe:

- **Deduplicate:** because the pipeline can run many times, the same record may arrive more than once. A window function keeps only the most recently extracted copy of each record.
- **Cast & rename:** every column is explicitly given the correct data type (text, number, date, timestamp), and a surrogate key is created where useful.

The dedup pattern (used in all four staging models): `row_number() over (partition by KEY order by extracted_at desc)` numbers the copies of each record newest-first, then `where row_num = 1` keeps only the newest. This is the single most important idea in the staging layer.

stg_locations.sql — one clean row per city

Deduplicates the city list to the latest geocoding result per city, then casts each attribute to its proper type (for example, coordinates to decimal numbers and population to a whole number).

```
with source as (

    select * from {{ source('open_meteo', 'raw_locations') }}

),

deduped as (

    select
        *,
        row_number() over (
            partition by location_id
            order by extracted_at desc
        ) as row_num

    from source

)

select
    cast(location_id as bigint) as location_id,
    cast(city_name as varchar) as city_name,
    cast(country as varchar) as country,
    cast(country_code as varchar) as country_code,
    cast(admin1 as varchar) as admin1,
    cast(latitude as double) as latitude,
    cast(longitude as double) as longitude,
    cast(timezone as varchar) as timezone,
    cast(elevation as double) as elevation,
    cast(population as bigint) as population,
    cast(extracted_at as timestamptz) as extracted_at

from deduped
where row_num = 1
```

stg_weather_daily.sql — clean observed daily weather

Same recipe, keyed on city *and* date. Note the first selected column: it builds a **surrogate key** (`weather_daily_id`) by joining the city id and the date with a dash — a single unique label for each city-day that later models and tests rely on.

```
with source as (

    select * from {{ source('open_meteo', 'raw_weather_daily') }}

),

deduped as (

    select
        *,
        row_number() over (
            partition by location_id, date
            order by extracted_at desc
        ) as row_num

    from source

)

select
    cast(location_id as varchar) || '-' || cast(date as varchar) as weather_daily_id,
    cast(source_name as varchar) as source_name,
    cast(location_id as bigint) as location_id,
    cast(city_name as varchar) as city_name,
    cast(country_code as varchar) as country_code,
    cast(date as date) as date,
    cast(latitude as double) as latitude,
    cast(longitude as double) as longitude,
    cast(timezone as varchar) as timezone,
    cast(temperature_2m_max as double) as temperature_2m_max,
    cast(temperature_2m_min as double) as temperature_2m_min,
    cast(temperature_2m_mean as double) as temperature_2m_mean,
    cast(precipitation_sum as double) as precipitation_sum,
    cast(rain_sum as double) as rain_sum,
    cast(snowfall_sum as double) as snowfall_sum,
    cast(wind_speed_10m_max as double) as wind_speed_10m_max,
    cast(extracted_at as timestamptz) as extracted_at

from deduped
where row_num = 1
```

stg_forecast_daily.sql — clean daily forecasts

Identical shape to the weather model, but for forecasts. Deduplicating on city + date keeps the **most recent forecast snapshot** for each future date, which is exactly what a user wants to see (“what does the latest forecast say?”).

```
with source as (

    select * from {{ source('open_meteo', 'raw_forecast_daily') }}

),

deduped as (

    select
        *,
        row_number() over (
            partition by location_id, date
            order by extracted_at desc
        ) as row_num

    from source

)

select
    cast(location_id as varchar) || '-' || cast(date as varchar) as forecast_daily_id,
```

```
    cast(source_name as varchar) as source_name,
    cast(location_id as bigint) as location_id,
    cast(city_name as varchar) as city_name,
    cast(country_code as varchar) as country_code,
    cast(date as date) as date,
    cast(latitude as double) as latitude,
    cast(longitude as double) as longitude,
    cast(timezone as varchar) as timezone,
    cast(temperature_2m_max as double) as temperature_2m_max,
    cast(temperature_2m_min as double) as temperature_2m_min,
    cast(temperature_2m_mean as double) as temperature_2m_mean,
    cast(precipitation_sum as double) as precipitation_sum,
    cast(rain_sum as double) as rain_sum,
    cast(snowfall_sum as double) as snowfall_sum,
    cast(wind_speed_10m_max as double) as wind_speed_10m_max,
    cast(extracted_at as timestamptz) as extracted_at

from deduped
where row_num = 1
```

stg_air_quality_hourly.sql — clean hourly air quality

The highest-volume source (one row per city per hour). It is deduplicated on city + timestamp and casts each pollutant plus the European AQI to numbers, ready to be summarised into daily figures later.

```
with source as (

    select * from {{ source('open_meteo', 'raw_air_quality_hourly') }}

),

deduped as (

    select
        *,
        row_number() over (
            partition by location_id, timestamp
            order by extracted_at desc
        ) as row_num

    from source

)

select
    cast(location_id as varchar) || '-' || cast(timestamp as varchar) as air_quality_hourly_id,
    cast(source_name as varchar) as source_name,
    cast(location_id as bigint) as location_id,
    cast(city_name as varchar) as city_name,
    cast(country_code as varchar) as country_code,
    cast(timestamp as timestamp) as timestamp,
    cast(latitude as double) as latitude,
    cast(longitude as double) as longitude,
    cast(timezone as varchar) as timezone,
    cast(pm10 as double) as pm10,
    cast(pm2_5 as double) as pm2_5,
    cast(carbon_monoxide as double) as carbon_monoxide,
    cast(nitrogen_dioxide as double) as nitrogen_dioxide,
    cast(ozone as double) as ozone,
    cast(european_aqi as integer) as european_aqi,
    cast(extracted_at as timestamptz) as extracted_at

from deduped
where row_num = 1
```

Testing the staging layer

The `_staging.yml` file documents each model and attaches automated tests. For example, every surrogate key is tested to be **unique** and **not null** — if a duplicate ever slipped through, `dbt build` would fail loudly.

```
  - name: stg_weather_daily
    description: >
      One row per city per day of actual (already-occurred) daily weather,
      deduplicated to the most recent extraction per location_id/date.
      Grain: location_id, date.
    columns:
      - name: weather_daily_id
        description: Surrogate key, location_id concatenated with date.
        data_tests:
          - unique
          - not_null
      - name: location_id
        description: Open-Meteo geocoding location identifier.
        data_tests:
          - not_null
      - name: date
        description: Local weather date.
        data_tests:
          - not_null
```

Excerpt: the test declarations for stg_weather_daily (lines 19–37 of _staging.yml).

8. Intermediate layer — business logic & enrichment

This is the “thinking” layer. Where staging only cleaned the data, the intermediate models start to **combine and interpret** it: joining weather to city details, rolling hourly air quality up into daily figures, and turning raw numbers into human-friendly flags. These models are built as real **tables** so the heavier calculations are computed once and stored.

int_city_day_weather.sql — weather + city attributes

Joins each day of observed weather to the details of the city it belongs to (country, coordinates, timezone, elevation, population). It also derives a small but useful new measure, `temperature_range` (the day's high minus its low).

```
-- Join actual daily weather to city/location attributes.
-- Grain: location_id, date.

with weather as (

    select * from {{ ref('stg_weather_daily') }}

),

locations as (

    select * from {{ ref('stg_locations') }}

)

select
    weather.weather_daily_id,
    weather.location_id,
    weather.date,

    -- location attributes
    locations.city_name,
    locations.country,
    locations.country_code,
    locations.admin1,
    locations.latitude,
    locations.longitude,
    locations.timezone,
    locations.elevation,
    locations.population,

    -- daily weather measures
    weather.temperature_2m_max,
    weather.temperature_2m_min,
    weather.temperature_2m_mean,
    weather.temperature_2m_max - weather.temperature_2m_min as temperature_range,
    weather.precipitation_sum,
    weather.rain_sum,
    weather.snowfall_sum,
    weather.wind_speed_10m_max,

    weather.extracted_at

from weather
left join locations
    on weather.location_id = locations.location_id
```

int_air_quality_daily.sql — hourly air quality rolled up to daily

Air quality arrives hour-by-hour, but the dashboard thinks in days. This model groups the hourly readings by city and date and computes daily averages and peaks for every pollutant. It also classifies the day's average European AQI into the official EEA band (good, fair, moderate, poor, very poor, extremely poor) and counts how many hours were actually observed, so partial days can be spotted.

```
-- Roll hourly air quality measurements up to one row per city per local date.
-- Grain: location_id, date.
--
-- Note: raw_air_quality_hourly is the highest-volume source. As history grows,
-- consider converting this model to an incremental materialization keyed on
-- `date` so daily rollups are not recomputed for the full history every run.

with hourly as (

    select * from {{ ref('stg_air_quality_hourly') }}

),

daily as (

    select
        location_id,
        cast(timestamp as date) as date,

        max(city_name) as city_name,
        max(country_code) as country_code,

        count(*) as hours_observed,
        count(european_aqi) as aqi_hours_observed,

        avg(european_aqi) as avg_european_aqi,
        max(european_aqi) as max_european_aqi,

        avg(pm2_5) as avg_pm2_5,
        max(pm2_5) as max_pm2_5,
        avg(pm10) as avg_pm10,
        max(pm10) as max_pm10,

        avg(carbon_monoxide) as avg_carbon_monoxide,
        avg(nitrogen_dioxide) as avg_nitrogen_dioxide,
        avg(ozone) as avg_ozone

    from hourly
    group by location_id, cast(timestamp as date)

)

select
    cast(location_id as varchar) || '-' || cast(date as varchar) as air_quality_daily_id,
    location_id,
    city_name,
    country_code,
    date,
    hours_observed,
    aqi_hours_observed,
    coalesce(hours_observed >= {{ var('complete_aqi_hours') }}, false) as is_complete_aqi_day,
    avg_european_aqi,
    max_european_aqi,

    -- European AQI band (EEA bands), classified on the daily average index.
    case
        when avg_european_aqi is null then null
        when avg_european_aqi < 20 then 'good'
        when avg_european_aqi < 40 then 'fair'
        when avg_european_aqi < 60 then 'moderate'
        when avg_european_aqi < 80 then 'poor'
```

```
        when avg_european_aqi < 100 then 'very poor'
        else 'extremely poor'
    end as european_aqi_band,

    avg_pm2_5,
    max_pm2_5,
    avg_pm10,
    max_pm10,
    avg_carbon_monoxide,
    avg_nitrogen_dioxide,
    avg_ozone
from daily
```

Key idea — the AQI band: the `case` statement converts an abstract index number into a word a person understands. An average AQI below 20 is “good”; 80 or above is “very poor”. This single derived column drives the coloured air-quality pills on the dashboard.

int_weather_flags.sql — turning numbers into plain-English flags

This is arguably the most important intermediate model. It takes the daily weather (plus the daily air quality) and produces a set of simple true/false flags: *is it rainy? hot? windy? freezing? snowy? is the air poor? is it, all things considered, a comfortable day?* Crucially, every threshold comes from the `vars` block in `dbt_project.yml` — so the definitions are tunable in one place.

```
-- Add analytical boolean flags by combining daily weather with daily air quality.
-- Grain: location_id, date.
--
-- Thresholds are defined as project vars (see dbt_project.yml) so they live in a
-- single, self-documenting place:
-- rainy day : precipitation_sum >= var('rainy_precip_mm')
-- hot day : temperature_2m_max >= var('hot_temp_c')
-- windy day : wind_speed_10m_max >= var('windy_kmh')
-- freezing day : temperature_2m_min <= var('freezing_temp_c')
-- snowy day : snowfall_sum > 0
-- poor AQI day : max_european_aqi >= var('poor_aqi')
-- comfortable day : mean temp within [comfortable_temp_min_c, comfortable_temp_max_c]
-- and none of the adverse flags

with weather as (

    select * from {{ ref('int_city_day_weather') }}

),

air_quality as (

    select
        location_id,
        date,
        avg_european_aqi,
        max_european_aqi
    from {{ ref('int_air_quality_daily') }}

),

joined as (

    select
        weather.weather_daily_id,
        weather.location_id,
        weather.date,
        weather.city_name,
        weather.country_code,

        weather.temperature_2m_max,
        weather.temperature_2m_min,
```

```
        weather.temperature_2m_mean,
        weather.precipitation_sum,
        weather.snowfall_sum,
        weather.wind_speed_10m_max,

        air_quality.avg_european_aqi,
        air_quality.max_european_aqi

    from weather
    left join air_quality
        on weather.location_id = air_quality.location_id
        and weather.date = air_quality.date

),

flagged as (

    select
        *,
        coalesce(precipitation_sum >= {{ var('rainy_precip_mm') }}, false) as is_rainy_day,
        coalesce(temperature_2m_max >= {{ var('hot_temp_c') }}, false) as is_hot_day,
        coalesce(wind_speed_10m_max >= {{ var('windy_kmh') }}, false) as is_windy_day,
        coalesce(temperature_2m_min <= {{ var('freezing_temp_c') }}, false) as is_freezing_day,
        coalesce(snowfall_sum > 0, false) as is_snowy_day,
        coalesce(max_european_aqi >= {{ var('poor_aqi') }}, false) as is_poor_aqi_day
    from joined

)

select
    weather_daily_id,
    location_id,
    date,
    city_name,
    country_code,

    temperature_2m_max,
    temperature_2m_min,
    temperature_2m_mean,
    precipitation_sum,
    snowfall_sum,
    wind_speed_10m_max,
    avg_european_aqi,
    max_european_aqi,

    is_rainy_day,
    is_hot_day,
    is_windy_day,
    is_freezing_day,
    is_snowy_day,
    is_poor_aqi_day,

    coalesce(
        not is_rainy_day
        and not is_hot_day
        and not is_windy_day
        and not is_freezing_day
        and not is_snowy_day
        and not is_poor_aqi_day
        and temperature_2m_mean >= {{ var('comfortable_temp_min_c') }}
        and temperature_2m_mean <= {{ var('comfortable_temp_max_c') }},
        false
    ) as is_comfortable_day

from flagged
```

The “comfortable day” rule: a day is comfortable only when the average temperature sits inside the pleasant band (18–26°C) *and* none of the bad flags (rainy, hot, windy, freezing, snowy, poor air) are set. The `coalesce(..., false)`

wrapper makes sure a missing value is treated as “not comfortable” rather than unknown.

int_forecast_daily_enriched.sql — forecasts + how far ahead they look

Enriches the forecasts with city attributes and, importantly, works out the **forecast horizon** : how many days ahead each forecast was looking (the gap between when the forecast was made and the date it predicts). It then buckets that into short / medium / long. This is what later lets the project measure “are 1-day forecasts more accurate than 7-day forecasts?”.

```
-- Enrich daily forecasts with location attributes and a forecast horizon.
-- Grain: location_id, date (the latest snapshot per forecast date, as produced by staging).
--
-- Snapshot logic: stg_forecast_daily keeps the most recently extracted forecast
-- per location_id/date. We surface the snapshot date (when the forecast was made)
-- and derive the forecast horizon, i.e. how many days ahead the forecast looked.

with forecast as (

    select * from {{ ref('stg_forecast_daily') }}

),

locations as (

    select * from {{ ref('stg_locations') }}

),

enriched as (

    select
        forecast.forecast_daily_id,
        forecast.location_id,
        forecast.date,

        -- snapshot logic
        forecast.extracted_at,
        cast(forecast.extracted_at as date) as forecast_snapshot_date,
        date_diff('day', cast(forecast.extracted_at as date), forecast.date) as forecast_horizon_days,

        -- location attributes
        locations.city_name,
        locations.country,
        locations.country_code,
        locations.admin1,
        locations.latitude,
        locations.longitude,
        locations.timezone,
        locations.elevation,
        locations.population,

        -- forecasted daily measures
        forecast.temperature_2m_max,
        forecast.temperature_2m_min,
        forecast.temperature_2m_mean,
        forecast.precipitation_sum,
        forecast.rain_sum,
        forecast.snowfall_sum,
        forecast.wind_speed_10m_max

    from forecast
    left join locations
        on forecast.location_id = locations.location_id

)

select
```

```
    *,
    case
        when forecast_horizon_days <= 3 then 'short (0-3d)'
        when forecast_horizon_days <= 7 then 'medium (4-7d)'
        else 'long (8d+)'
    end as forecast_horizon_bucket
from enriched
```

9. Marts layer — the analytics-ready outputs

The marts are the finished products. Everything the dashboard shows comes from this layer. There are three kinds: a **dimension** (`dim_location`, the master city list), two **fact** tables (the detailed measurements), and two **marts** (pre-scored, ready-to-display views).

dim_location.sql — the master list of cities

A clean, single-purpose list of every tracked city and its attributes. In data-warehouse terms this is a **dimension**: descriptive information that the fact tables point to. Keeping it separate avoids repeating city details in every row of every fact table.

```
-- One row per tracked city. Source of truth for location attributes
-- referenced by all fact tables. Grain: location_id.

with source as (

    select * from {{ ref('stg_locations') }}

)

select
    location_id,
    city_name,
    country,
    country_code,
    admin1,
    latitude,
    longitude,
    timezone,
    elevation,
    population
from source
```

fct_city_weather_day.sql — the complete observed-weather fact

The main **fact table**: one row per city per observed day, bringing together everything from the intermediate layer — all the weather measures, all the condition flags, and the daily air quality — into one wide, convenient table. This is what the dashboard reads for historical trends.

```
-- One row per city per day of observed weather, combining all weather measures,
-- analytical flags and air quality. Grain: location_id, date.

with weather as (

    select * from {{ ref('int_city_day_weather') }}

),

flags as (

    select
        weather_daily_id,
        is_rainy_day,
        is_hot_day,
        is_windy_day,
        is_freezing_day,
        is_snowy_day,
        is_poor_aqi_day,
        is_comfortable_day
    from {{ ref('int_weather_flags') }}

),
```

```
air_quality as (

    select * from {{ ref('int_air_quality_daily') }}

),

final as (

    select
        -- keys
        weather.weather_daily_id,
        weather.location_id,
        weather.date,

        -- location (FK to dim_location)
        weather.city_name,
        weather.country,
        weather.country_code,
        weather.admin1,
        weather.latitude,
        weather.longitude,
        weather.timezone,
        weather.elevation,
        weather.population,

        -- weather measures
        weather.temperature_2m_max,
        weather.temperature_2m_min,
        weather.temperature_2m_mean,
        weather.temperature_range,
        weather.precipitation_sum,
        weather.rain_sum,
        weather.snowfall_sum,
        weather.wind_speed_10m_max,

        -- analytical flags
        flags.is_rainy_day,
        flags.is_hot_day,
        flags.is_windy_day,
        flags.is_freezing_day,
        flags.is_snowy_day,
        flags.is_poor_aqi_day,
        flags.is_comfortable_day,

        -- air quality
        air_quality.avg_european_aqi,
        air_quality.max_european_aqi,
        air_quality.european_aqi_band,
        air_quality.is_complete_aqi_day,
        air_quality.avg_pm2_5,
        air_quality.max_pm2_5,
        air_quality.avg_pm10,
        air_quality.max_pm10,
        air_quality.avg_carbon_monoxide,
        air_quality.avg_nitrogen_dioxide,
        air_quality.avg_ozone,

        weather.extracted_at

    from weather
    left join flags
        on weather.weather_daily_id = flags.weather_daily_id
    left join air_quality
        on weather.location_id = air_quality.location_id
        and weather.date = air_quality.date

)

select * from final
```

fct_forecast_accuracy.sql — did the forecast get it right?

This model lines up each forecast against what actually happened and pre-computes the **error** for every measure — both signed (was the forecast too high or too low?) and absolute (how far off, regardless of direction?). Rows for future dates simply have no actuals yet, flagged by `has_actuals = false`. Pre-computing errors here means the dashboard can instantly show forecast reliability without re-doing the maths.

```
-- Compares the latest forecast snapshot for each city/date against actual
-- observed weather, producing signed and absolute errors per measure.
-- Rows where has_actuals = false represent future-dated forecasts with no
-- observed counterpart yet. Grain: location_id, date.

with forecast as (

    select * from {{ ref('int_forecast_daily_enriched') }}

),

actuals as (

    select
        location_id,
        date,
        temperature_2m_max as actual_temperature_2m_max,
        temperature_2m_min as actual_temperature_2m_min,
        temperature_2m_mean as actual_temperature_2m_mean,
        precipitation_sum as actual_precipitation_sum,
        rain_sum as actual_rain_sum,
        snowfall_sum as actual_snowfall_sum,
        wind_speed_10m_max as actual_wind_speed_10m_max
    from {{ ref('int_city_day_weather') }}

),

final as (

    select
        -- surrogate key
        cast(forecast.location_id as varchar) || '-' || cast(forecast.date as varchar) as forecast_accuracy_id,

        forecast.forecast_daily_id,
        forecast.location_id,
        forecast.date,

        -- location (FK to dim_location)
        forecast.city_name,
        forecast.country,
        forecast.country_code,

        -- forecast metadata
        forecast.forecast_snapshot_date,
        forecast.forecast_horizon_days,
        forecast.forecast_horizon_bucket,

        -- forecasted values
        forecast.temperature_2m_max as fcst_temperature_2m_max,
        forecast.temperature_2m_min as fcst_temperature_2m_min,
        forecast.temperature_2m_mean as fcst_temperature_2m_mean,
        forecast.precipitation_sum as fcst_precipitation_sum,
        forecast.rain_sum as fcst_rain_sum,
        forecast.snowfall_sum as fcst_snowfall_sum,
        forecast.wind_speed_10m_max as fcst_wind_speed_10m_max,

        -- actual values
        actuals.actual_temperature_2m_max,
        actuals.actual_temperature_2m_min,
        actuals.actual_temperature_2m_mean,
        actuals.actual_precipitation_sum,
```

```
        actuals.actual_rain_sum,
        actuals.actual_snowfall_sum,
        actuals.actual_wind_speed_10m_max,

        -- signed errors (forecast − actual; positive = over-forecast)
        forecast.temperature_2m_max - actuals.actual_temperature_2m_max as temp_max_error,
        forecast.temperature_2m_min - actuals.actual_temperature_2m_min as temp_min_error,
        forecast.temperature_2m_mean - actuals.actual_temperature_2m_mean as temp_mean_error,
        forecast.precipitation_sum - actuals.actual_precipitation_sum as precipitation_error,
        forecast.wind_speed_10m_max - actuals.actual_wind_speed_10m_max as wind_speed_error,

        -- absolute errors (useful for MAE aggregations)
        abs(forecast.temperature_2m_max - actuals.actual_temperature_2m_max) as temp_max_abs_error,
        abs(forecast.temperature_2m_min - actuals.actual_temperature_2m_min) as temp_min_abs_error,
        abs(forecast.temperature_2m_mean - actuals.actual_temperature_2m_mean) as temp_mean_abs_error,
        abs(forecast.precipitation_sum - actuals.actual_precipitation_sum) as precipitation_abs_error,
        abs(forecast.wind_speed_10m_max - actuals.actual_wind_speed_10m_max) as wind_speed_abs_error,

        actuals.actual_temperature_2m_max is not null as has_actuals

    from forecast
    left join actuals
        on forecast.location_id = actuals.location_id
        and forecast.date = actuals.date

)

select * from final
```

Key idea — signed vs. absolute error: `forecast − actual` keeps the direction (a positive number means the forecast was too warm), while `abs(forecast − actual)` drops the sign so the errors can be averaged into a Mean Absolute Error (MAE) — the standard way to summarise forecast quality.

mart_latest_conditions.sql — current snapshot + the visit score

This is the powerhouse behind the Overview and Recommendations pages. For every city it combines: the most recent observed day, today's and tomorrow's forecast, and — most importantly — a **7-day composite visit score (0–100)**. It is built as a **view** so that `current_date` is always evaluated fresh ("today" is really today). It even generates a short plain-English recommendation reason.

```
{{ config(materialized='view') }}

-- One row per city. Current observed snapshot + today/tomorrow forecast
-- + 7-day visit scores. Evaluated at query time (view) so current_date is live.

with latest_observed as (

    select *
    from {{ ref('fct_city_weather_day') }}
    qualify row_number() over (partition by location_id order by date desc) = 1

),

upcoming as (

    select *
    from {{ ref('fct_forecast_accuracy') }}
    where date >= current_date

),

today_fcst as (

    select *
    from {{ ref('fct_forecast_accuracy') }}
    where date = current_date
```

```
),

tomorrow_fcst as (

    select *
    from {{ ref('fct_forecast_accuracy') }}
    where date = current_date + interval '1 day'

),

score_agg as (

    select
        location_id,

        round(avg(
            0.40 * (1 - least(abs(coalesce(fcst_temperature_2m_mean, 22.0) - 22.0) / 20.0, 1.0))
            + 0.35 * (1 - least(coalesce(fcst_precipitation_sum, 0) / 20.0, 1.0))
            + 0.25 * (1 - least(coalesce(fcst_wind_speed_10m_max, 0) / 80.0, 1.0))
        ) * 100, 1) as visit_score,

        round(avg(
            1 - least(abs(coalesce(fcst_temperature_2m_mean, 22.0) - 22.0) / 20.0, 1.0)
        ) * 100, 1) as avg_temp_comfort_score,

        round(avg(
            1 - least(coalesce(fcst_precipitation_sum, 0) / 20.0, 1.0)
        ) * 100, 1) as avg_rain_score,

        round(avg(
            1 - least(coalesce(fcst_wind_speed_10m_max, 0) / 80.0, 1.0)
        ) * 100, 1) as avg_wind_score

    from upcoming
    group by location_id

),

final as (

    select
        -- location
        l.location_id,
        l.city_name,
        l.country,
        l.country_code,
        l.latitude,
        l.longitude,
        l.timezone,
        l.elevation,
        l.population,

        -- latest observed
        o.date as last_observed_date,
        o.temperature_2m_max as obs_temp_max,
        o.temperature_2m_min as obs_temp_min,
        o.temperature_2m_mean as obs_temp_mean,
        o.precipitation_sum as obs_precip,
        o.wind_speed_10m_max as obs_wind,
        o.is_rainy_day,
        o.is_hot_day,
        o.is_windy_day,
        o.is_freezing_day,
        o.is_snowy_day,
        o.is_poor_aqi_day,
        o.is_comfortable_day,
        o.avg_european_aqi,
        o.max_european_aqi,
```

```
        o.european_aqi_band,

        -- today's forecast
        t.fcst_temperature_2m_max as today_temp_max,
        t.fcst_temperature_2m_min as today_temp_min,
        t.fcst_temperature_2m_mean as today_temp_mean,
        t.fcst_precipitation_sum as today_precip,
        t.fcst_wind_speed_10m_max as today_wind,

        -- tomorrow's forecast
        tm.fcst_temperature_2m_max as tomorrow_temp_max,
        tm.fcst_temperature_2m_min as tomorrow_temp_min,
        tm.fcst_precipitation_sum as tomorrow_precip,

        -- composite visit score (7-day horizon)
        vs.visit_score,
        vs.avg_temp_comfort_score,
        vs.avg_rain_score,
        vs.avg_wind_score,

        case
            when o.is_comfortable_day then 'Comfortable'
            when o.is_hot_day then 'Hot'
            when o.is_rainy_day then 'Rainy'
            when o.is_windy_day then 'Windy'
            when o.is_freezing_day then 'Freezing'
            when o.is_snowy_day then 'Snowy'
            else 'Mixed'
        end as condition_label,

        case
            when coalesce(vs.visit_score, 0) >= 75 then
                case
                    when o.is_comfortable_day then 'warm, dry, comfortable'
                    when not o.is_rainy_day and not o.is_windy_day then 'clear skies, light wind'
                    else 'good overall conditions'
                end
            when coalesce(vs.visit_score, 0) >= 50 then 'decent conditions expected'
            else 'challenging weather ahead'
        end as recommendation_reason

    from {{ ref('dim_location') }} l
    left join latest_observed o on l.location_id = o.location_id
    left join today_fcst t on l.location_id = t.location_id
    left join tomorrow_fcst tm on l.location_id = tm.location_id
    left join score_agg vs on l.location_id = vs.location_id

)

select * from final
```

mart_forecast_upcoming.sql — the reweightable forecast mart

One row per city per upcoming day. Alongside the forecast figures it stores the three **score components** separately (temperature comfort, low rain, low wind). Because the components are stored individually, the Recommendations page can let a user re-weight them live (“I care more about sunshine than wind”) without asking the database again.

```
{{ config(materialized='view') }}

-- One row per city per upcoming forecast date (>= today).
-- Includes pre-computed score components so the dashboard can reweight live.

with forecast as (

    select *
    from {{ ref('fct_forecast_accuracy') }}
```

```
    where date >= current_date

),

locations as (

    select location_id, latitude, longitude, timezone
    from {{ ref('dim_location') }}

)

select
    f.forecast_accuracy_id,
    f.location_id,
    f.city_name,
    f.country,
    f.country_code,
    f.date,
    f.forecast_snapshot_date,
    f.forecast_horizon_days,
    f.forecast_horizon_bucket,

    l.latitude,
    l.longitude,
    l.timezone,

    f.fcst_temperature_2m_max,
    f.fcst_temperature_2m_min,
    f.fcst_temperature_2m_mean,
    f.fcst_precipitation_sum,
    f.fcst_rain_sum,
    f.fcst_snowfall_sum,
    f.fcst_wind_speed_10m_max,

    -- score components (0-100); stored so dashboard can reweight without re-querying
    round(
        (1 - least(abs(coalesce(f.fcst_temperature_2m_mean, 22.0) - 22.0) / 20.0, 1.0)) * 100
    , 1) as temp_comfort_score,

    round(
        (1 - least(coalesce(f.fcst_precipitation_sum, 0) / 20.0, 1.0)) * 100
    , 1) as rain_score,

    round(
        (1 - least(coalesce(f.fcst_wind_speed_10m_max, 0) / 80.0, 1.0)) * 100
    , 1) as wind_score,

    -- composite visit score with default weights (40 / 35 / 25)
    round((
        0.40 * (1 - least(abs(coalesce(f.fcst_temperature_2m_mean, 22.0) - 22.0) / 20.0, 1.0))
        + 0.35 * (1 - least(coalesce(f.fcst_precipitation_sum, 0) / 20.0, 1.0))
        + 0.25 * (1 - least(coalesce(f.fcst_wind_speed_10m_max, 0) / 80.0, 1.0))
    ) * 100, 1) as visit_score,

    case
        when coalesce(f.fcst_temperature_2m_max, 0) >= 30 then 'Hot'
        when coalesce(f.fcst_precipitation_sum, 0) >= 5 then 'Rainy'
        when coalesce(f.fcst_wind_speed_10m_max, 0) >= 38 then 'Windy'
        when coalesce(f.fcst_temperature_2m_min, 0) <= 0 then 'Freezing'
        when coalesce(f.fcst_snowfall_sum, 0) > 0 then 'Snowy'
        when coalesce(f.fcst_temperature_2m_mean, 20) between 18 and 26
             and coalesce(f.fcst_precipitation_sum, 0) < 1 then 'Comfortable'
        else 'Mixed'
    end as condition_label,

    f.has_actuals

from forecast f
left join locations l on f.location_id = l.location_id
```

10. Model lineage map — what feeds what

“Lineage” is the family tree of the data: which models depend on which. dbt understands this automatically from the `ref()` and `source()` functions, and always builds things in the correct order. The chart below traces the flow from the four raw sources on the left to the dashboard-facing marts on the right.

Figure 2 — Simplified dependency graph. Arrows point from a source to the model that reads it.

Dependency table

The same information, model by model. “Reads from” lists a model's direct inputs; “used by” lists who depends on it.

Model	Layer	Reads from	Used by
`stg_locations`	staging	raw_locations	int_city_day_weather, int_forecast_daily_enriched, dim_location
`stg_weather_daily`	staging	raw_weather_daily	int_city_day_weather
`stg_forecast_daily`	staging	raw_forecast_daily	int_forecast_daily_enriched
`stg_air_quality_hourly`	staging	raw_air_quality_hourly	int_air_quality_daily
`int_city_day_weather`	intermediate	stg_weather_daily, stg_locations	int_weather_flags, fct_city_weather_day, fct_forecast_accuracy
`int_air_quality_daily`	intermediate	stg_air_quality_hourly	int_weather_flags, fct_city_weather_day
`int_weather_flags`	intermediate	int_city_day_weather, int_air_quality_daily	fct_city_weather_day
`int_forecast_daily_enriched`	intermediate	stg_forecast_daily, stg_locations	fct_forecast_accuracy
`dim_location`	marts	stg_locations	fct tables (relationship tests), marts
`fct_city_weather_day`	marts	int_city_day_weather, int_weather_flags, int_air_quality_daily	mart_latest_conditions, dashboard history
`fct_forecast_accuracy`	marts	int_forecast_daily_enriched, int_city_day_weather	mart_latest_conditions, mart_forecast_upcoming
`mart_latest_conditions`	marts	fct_city_weather_day, fct_forecast_accuracy, dim_location	Overview, Recommendations, City detail

Model	Layer	Reads from	Used by
mart_forecast_upcoming	marts	fct_forecast_accuracy, dim_location	Map, Forecast, Comparison, Recommendations

11. The visit score explained

The **visit score** is the single number at the centre of the whole product: a 0–100 rating of how good a city's weather is for a holiday. It appears on almost every page. Here is exactly how it is built — the same formula lives in `mart_latest_conditions` and `mart_forecast_upcoming`.

Step 1 — three component scores, each 0 to 100

- **Temperature comfort (40%):** best when the average temperature is around 22°C. The further from 22°C, the lower the score, hitting 0 once you are 20° away in either direction.
- **Low rain (35%):** 100 when there is no rain, falling to 0 at 20 mm or more of precipitation.
- **Low wind (25%):** 100 when calm, falling to 0 at 80 km/h or more of wind.

Step 2 — combine with weights

The three components are blended using their weights (40% / 35% / 25%) and multiplied by 100. The `least(..., 1.0)` function caps each penalty so no component can ever drag the score below 0, and `coalesce` substitutes a sensible default when a value is missing.

```
-- composite visit score with default weights (40 / 35 / 25)
round((
    0.40 * (1 - least(abs(coalesce(f.fcst_temperature_2m_mean, 22.0) - 22.0) / 20.0, 1.0))
    + 0.35 * (1 - least(coalesce(f.fcst_precipitation_sum, 0) / 20.0, 1.0))
    + 0.25 * (1 - least(coalesce(f.fcst_wind_speed_10m_max, 0) / 80.0, 1.0))
) * 100, 1) as visit_score,
```

The composite score, exactly as written in mart_forecast_upcoming.sql (lines 57–62).

Step 3 — a worked example

Suppose a forecast day has an average temperature of 24°C, 2 mm of rain and 15 km/h of wind. The score is calculated like this:

Component	Value	Component score	Weight	Contribution
Temperature comfort	24°C	1 − (2/20) = 90.0	40%	36.0
Low rain	2 mm	1 − (2/20) = 90.0	35%	31.5
Low wind	15 km/h	1 − (15/80) = 81.3	25%	20.3
Visit score				**87.8 / 100**

Finally, `mart_latest_conditions` **averages this daily score across the next 7 forecast days**, giving each city one headline number that reflects the whole upcoming week rather than a single day. On the dashboard, a score of 70+ is shown as green (good), 45–69 as amber, and below 45 as red.

12. Data-quality tests

A major benefit of dbt is that data quality can be tested automatically, just like software. When `dbt build` runs, it not only rebuilds the tables but also checks dozens of rules; if any fail, the build stops and reports the problem. There are two kinds here.

Generic tests (declared in the .yml files)

These are reusable, one-line rules attached to columns. The project uses four types:

- **unique** — no duplicate values (used on every surrogate key).
- **not_null** — the value must always be present (used on keys, dates, city names).
- **relationships** — every `location_id` in a fact must exist in the location list (no "orphan" rows).
- **dbt_utils.unique_combination_of_columns** — proves the grain, e.g. that (city, date) never repeats.

Singular tests (custom SQL in tests/)

These are hand-written checks. Each is simply a query that finds "bad" rows — if it returns any rows at all, the test fails. Both are short and easy to read.

assert_city_weather_temp_in_range.sql

Catches physically impossible temperatures (outside −90°C to 60°C) or a maximum lower than the minimum — a classic sign of a data error.

```
-- Fails if any row contains temperature values outside physically plausible
-- global bounds (-90 °C to 60 °C) or if max < min (impossible range).

select
    weather_daily_id,
    location_id,
    date,
    temperature_2m_max,
    temperature_2m_min
from {{ ref('fct_city_weather_day') }}
where
    temperature_2m_max > 60
    or temperature_2m_min < -90
    or temperature_2m_max < temperature_2m_min
```

assert_forecast_horizon_non_negative.sql

A forecast should predict the future, so the horizon (days ahead) can never be negative. If it were, it would mean a forecast was somehow recorded *after* the day it was predicting.

```
-- Fails if any forecast horizon is negative, which would mean the forecast
-- snapshot was recorded after the date it was predicting.

select
    forecast_accuracy_id,
    location_id,
    date,
    forecast_snapshot_date,
    forecast_horizon_days
from {{ ref('fct_forecast_accuracy') }}
where forecast_horizon_days < 0
```

13. The extraction & load pipeline (Python)

Before dbt can transform anything, the raw data has to arrive. Three Python scripts handle this. They are deliberately small and use only well-tested tools.

scripts/extract_open_meteo.py — downloading the data

This script contacts the Open-Meteo API and saves four CSV files. It looks up each city's coordinates, then requests recent weather, upcoming forecasts and hourly air quality. It is written defensively: it retries automatically if the internet request fails, and it politely pauses between calls.

The cities and measures it collects

```
DEFAULT_CITIES = [
    "Madrid",
    "Lisbon",
    "Paris",
    "Berlin",
    "Amsterdam",
    "Brussels",
    "London",
    "Rome",
]
DEFAULT_DAILY_WEATHER_VARIABLES = [
    "temperature_2m_max",
    "temperature_2m_min",
    "temperature_2m_mean",
    "precipitation_sum",
    "rain_sum",
    "snowfall_sum",
    "wind_speed_10m_max",
]
DEFAULT_AIR_QUALITY_VARIABLES = [
    "pm10",
    "pm2_5",
    "carbon_monoxide",
    "nitrogen_dioxide",
    "ozone",
    "european_aqi",
]
```

Reliable downloading with automatic retries

If a request fails with a temporary error (like “server busy”), the script waits a little longer each time and tries again — up to three attempts — instead of giving up.

```
def get_json(url: str, params: dict[str, Any]) -> dict[str, Any]:
    query_string = urlencode(params, doseq=True)
    request_url = f"{url}?{query_string}"
    last_error: Exception | None = None

    for attempt in range(1, MAX_HTTP_ATTEMPTS + 1):
        request = Request(
            request_url,
            headers={"User-Agent": "dbt-ie-open-meteo-assignment/1.0"},
        )

        try:
            with urlopen(
                request,
                timeout=REQUEST_TIMEOUT_SECONDS,
                context=get_ssl_context(),
            ) as response:
                payload = response.read().decode("utf-8")
            return json.loads(payload)
```

```
        except HTTPError as exc:
            last_error = exc
            should_retry = exc.code in RETRY_STATUS_CODES
            if not should_retry or attempt == MAX_HTTP_ATTEMPTS:
                raise RuntimeError(
                    f"Open-Meteo request failed with HTTP {exc.code}: {request_url}"
                ) from exc
        except (TimeoutError, URLError, json.JSONDecodeError) as exc:
            last_error = exc
            if attempt == MAX_HTTP_ATTEMPTS:
                raise RuntimeError(
                    f"Open-Meteo request failed after {attempt} attempts: {request_url}"
                ) from exc

        wait_seconds = min(2 ** (attempt - 1), 10)
        print(
            f"Request failed on attempt {attempt}; retrying in {wait_seconds}s",
            file=sys.stderr,
        )
        time.sleep(wait_seconds)

    raise RuntimeError(f"Open-Meteo request failed: {request_url}") from last_error
```

The main loop, city by city

```
    for city in args.cities:
        print(f"Extracting {city}...", file=sys.stderr)
        location = geocode_city(city)
        locations.append({**location, "extracted_at": extracted_at})
        time.sleep(args.pause_seconds)

        weather_daily_rows.extend(
            fetch_recent_weather(location, args.past_days, extracted_at)
        )
        time.sleep(args.pause_seconds)

        forecast_daily_rows.extend(fetch_forecast(location, args.forecast_days, extracted_at))
        time.sleep(args.pause_seconds)

        air_quality_hourly_rows.extend(fetch_air_quality(location, extracted_at))
        time.sleep(args.pause_seconds)

    write_csv(output_dir / "raw_locations.csv", locations)
    write_csv(output_dir / "raw_weather_daily.csv", weather_daily_rows)
    write_csv(output_dir / "raw_forecast_daily.csv", forecast_daily_rows)
    write_csv(output_dir / "raw_air_quality_hourly.csv", air_quality_hourly_rows)
```

scripts/load_raw_to_motherduck.py — loading the CSVs

This script reads the four CSV files and loads them into the raw schema in MotherDuck. Its cleverest feature is a safe **upsert**: by default it updates existing records and adds new ones (keyed on a natural key), so the pipeline can run repeatedly without creating duplicates. A `--replace` flag is available to wipe and reload instead.

Each table's natural key is defined once

```
TABLE_SPECS = (
    TableSpec(
        file_name="raw_locations.csv",
        table_name="raw_locations",
        columns=LOCATION_COLUMNS,
        key_columns=("location_id",),
    ),
    TableSpec(
        file_name="raw_weather_daily.csv",
        table_name="raw_weather_daily",
        columns=DAILY_WEATHER_COLUMNS,
        key_columns=("location_id", "date"),
    ),
    TableSpec(
        file_name="raw_forecast_daily.csv",
        table_name="raw_forecast_daily",
        columns=DAILY_WEATHER_COLUMNS,
        key_columns=("location_id", "date", "extracted_at"),
    ),
    TableSpec(
        file_name="raw_air_quality_hourly.csv",
        table_name="raw_air_quality_hourly",
        columns=AIR_QUALITY_COLUMNS,
        key_columns=("location_id", "timestamp"),
    ),
)
```

The upsert: delete matching rows, then insert the fresh ones

```
def delete_existing_rows(
    connection: duckdb.DuckDBPyConnection,
    spec: TableSpec,
    replace: bool,
) -> None:
    relation = table_relation(spec)
    if replace:
        connection.execute(f"delete from {relation}")
        return

    conditions = []
    for key_column in spec.key_columns:
        data_type = column_type(spec, key_column)
        key_identifier = quote_identifier(key_column)
        conditions.append(
            f"target.{key_identifier} = "
            f"try_cast(nullif(incoming.{key_identifier}, '') as {data_type})"
        )

    connection.execute(
        f"""
        delete from {relation} as target
        using {temp_view_name(spec)} as incoming
        where {' and '.join(conditions)}
        """
    )
```

scripts/bootstrap_motherduck.py — first-time setup

A helper that runs the setup SQL and then **verifies** the result — confirming the four schemas and four raw tables all exist, and failing clearly if any are missing. It is safe to run any number of times.

```
def run_sql_file(
    connection: duckdb.DuckDBPyConnection,
    sql_file: Path,
    database: str,
) -> None:
```

```
    sql = sql_file.read_text(encoding="utf-8").replace(DEFAULT_DATABASE, database)
    for statement in sql.split(";"):
        statement = statement.strip()
        if statement:
            connection.execute(statement)
```

14. The Streamlit dashboard

The dashboard is the face of the project — a web app with six tabs. It reads only from the marts layer, and it is built entirely in Python (even the styling). The design follows a strict rulebook (see `AGENT.md`): a sleek fintech look, one cobalt accent colour, no emojis, and a light/dark toggle.

dashboard/app.py — the entry point

Small but central: it sets up the page, applies the theme once, and lays out the six tabs. Each tab delegates to its own page module by calling that module's `show()` function.

```
import streamlit as st

st.set_page_config(
    page_title="Weather Analytics",
    page_icon="  ",
    layout="wide",
    initial_sidebar_state="collapsed",
)

import sys, os
sys.path.insert(0, os.path.dirname(__file__))
from utils import theme as rv

rv.init_theme()
rv.inject_css()

from pages import overview, map_view, forecast, comparison, recommendations, city_detail

PAGES = [
    ("Overview", overview),
    ("Map", map_view),
    ("Forecast & trends", forecast),
    ("Comparison", comparison),
    ("Recommendations", recommendations),
    ("City detail", city_detail),
]

brand_col, ctrl_col = st.columns([9, 1.1])
brand_col.markdown(
    '<span class="rv-brand">Weather analytics</span>'
    '<span class="rv-tagline">Find the ideal place for your ideal holiday</span>',
    unsafe_allow_html=True,
)
with ctrl_col:
    with st.container(key="hdr_ctrls"):
        toggle_col, fs_col = st.columns(2, gap="small")
        rv.render_toggle(toggle_col)
        rv.render_fullscreen(fs_col)

tabs = st.tabs([label for label, _ in PAGES])
for tab, (_, module) in zip(tabs, PAGES):
    with tab:
        module.show()
```

dashboard/utils/db.py — talking to the warehouse

Every database read goes through this one file. Two design choices make it safe and fast: the connection is **read-only** (the dashboard can never change the data), and results are **cached** for five minutes (`@st.cache_data(ttl=300)`) so repeated views are instant and don't hammer the database.

```
import os
import duckdb
import pandas as pd
```

```
import streamlit as st
from dotenv import load_dotenv

load_dotenv()

SCHEMA = "marts"


def _conn_str() -> str:
    token = os.getenv("MOTHERDUCK_TOKEN", "")
    db = os.getenv("MOTHERDUCK_DATABASE", "open_meteo_europe_sa")
    if token:
        return f"md:{db}?motherduck_token={token}"
    return os.path.join(os.path.dirname(__file__), "..", "..", "my_database.duckdb")


@st.cache_data(ttl=300, show_spinner=False)
def run_query(sql: str) -> pd.DataFrame:
    conn = duckdb.connect(_conn_str(), read_only=True)
    try:
        return conn.execute(sql).df()
    finally:
        conn.close()


def latest_conditions() -> pd.DataFrame:
    return run_query(f"SELECT * FROM {SCHEMA}.mart_latest_conditions ORDER BY city_name")


def forecast_upcoming() -> pd.DataFrame:
    return run_query(
        f"SELECT * FROM {SCHEMA}.mart_forecast_upcoming ORDER BY city_name, date"
    )


def weather_history(days: int = 30) -> pd.DataFrame:
    return run_query(f"""
        SELECT *
        FROM {SCHEMA}.fct_city_weather_day
        WHERE date >= current_date - INTERVAL '{days} days'
        ORDER BY city_name, date
    """)


def forecast_accuracy_summary() -> pd.DataFrame:
    return run_query(f"""
        SELECT
            city_name,
            forecast_horizon_bucket,
            round(avg(temp_mean_abs_error), 2) AS mae_temp_mean,
            round(avg(temp_max_abs_error), 2) AS mae_temp_max,
            round(avg(precipitation_abs_error), 2) AS mae_precip,
            round(avg(wind_speed_abs_error), 2) AS mae_wind,
            count(*) AS n
        FROM {SCHEMA}.fct_forecast_accuracy
        WHERE has_actuals
        GROUP BY city_name, forecast_horizon_bucket
        ORDER BY city_name, forecast_horizon_bucket
    """)


def dim_location() -> pd.DataFrame:
    return run_query(f"SELECT * FROM {SCHEMA}.dim_location ORDER BY city_name")
```

dashboard/utils/theme.py — one place for all styling

All colours, fonts, and reusable visual components live here, so the whole app stays consistent. It defines the design tokens for light and dark mode, and helper functions like `condition_pill()` and `score_bar()` that build small HTML snippets reused across pages. The excerpt below shows the colour tokens and the rule that maps a score to a colour.

```
def score_role(score):
    if not _is_num(score):
        return "neutral"
    if score >= 70:
        return "good"
    if score >= 45:
        return "warn"
    return "bad"


def _pill_colors(role):
    dbg, dtx, lbg, ltx, _ = ROLES.get(role, ROLES["neutral"])
    return (dbg, dtx) if st.session_state.get("theme") == "dark" else (lbg, ltx)


def condition_pill(condition):
    """Line-icon + colored condition pill (reused across pages)."""
    condition = condition or "Mixed"
    icon, role = CONDITIONS.get(condition, ("ti-cloud", "neutral"))
    bg, tx = _pill_colors(role)
    return (f'<span class="rv-pill" style="background:{bg};color:{tx}">'
            f'<i class="ti {icon}"></i>{condition}</span>')


def score_bar(score):
    """A visit-score cell: a colored progress bar plus the value (– if missing)."""
    if not _is_num(score):
        return '<div class="rv-sc"><div class="rv-scbar"></div><span>–</span></div>'
    color = ROLES[score_role(score)][4]
    width = max(0, min(100, score))
    return (f'<div class="rv-sc"><div class="rv-scbar">'
            f'<span style="width:{width:.0f}%;background:{color}"></span></div>'
            f'<span>{score:.0f}</span></div>')
```

Excerpt: theme.py (lines 234–266) — the score-to-colour logic and the reusable pill / bar builders.

The six pages

Each file in `dashboard/pages/` renders one tab. They all read from the same cached mart queries and share the theme.

Page file	Tab	What it shows	Reads
overview.py	Overview	KPI cards for every city plus a ‘top pick’ hero	mart_latest_conditions
map_view.py	Map	Cities on an interactive map, sized/coloured by a variable	mart_forecast_upcoming, dim_location
forecast.py	Forecast & trends	Multi-city forecast line charts (up to 4 cities)	mart_forecast_upcoming
comparison.py	Comparison	Side-by-side radar / line + metric table (up to 6)	mart_forecast_upcoming, history
recommendations.py	Recommendations	Re-weightable leaderboard with activity presets	mart_forecast_upcoming
city_detail.py	City detail	One city in depth: stats, 7-day cards, trends	all three marts

Overview page — the top-pick logic

The full Overview page is compact. Its neat trick: it finds the highest-scoring city, shows it as a big hero banner, and **removes it from the grid below** so it is not shown twice — and because this is keyed on the score, the excluded city updates automatically as data changes.

```
import streamlit as st
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from utils.db import latest_conditions, forecast_upcoming
from utils import theme as rv


def show():
    with st.spinner("Loading current conditions…"):
        lc = latest_conditions()
        fcst = forecast_upcoming()

    if lc.empty:
        st.warning("No data available. Run `dbt build` to build the mart models.")
        return

    top = lc.sort_values("visit_score", ascending=False).iloc[0]
    # Exclude the current top pick from the grid; it already headlines the hero.
    # Keyed off visit_score, so the excluded city updates automatically on reload.
    rest = lc.drop(index=top.name)
    cards = "".join(rv.kpi_card(row, fcst) for _, row in rest.iterrows())

    st.markdown(
        rv.hero(top)
        + '<div class="rv-section">Current conditions by city</div>'
        + f'<div class="rv-grid">{cards}</div>',
        unsafe_allow_html=True,
    )
```

Recommendations page — live re-weighting

This page lets the user pick an activity (Beach day, Hiking, City walk, Skiing), which sets the temperature/rain/wind weights, then recomputes every city's score in the browser using the pre-stored component scores. The core recompute is just a weighted average:

```
PRESETS = {
    "Custom": None,
    "Beach day": dict(w_temp=0.50, w_rain=0.30, w_wind=0.20),
    "Hiking": dict(w_temp=0.40, w_rain=0.35, w_wind=0.25),
    "City walk": dict(w_temp=0.35, w_rain=0.45, w_wind=0.20),
    "Skiing": dict(w_temp=0.20, w_rain=0.45, w_wind=0.35),
}


def _segmented(label, options, key):
    if hasattr(st, "segmented_control"):
        return st.segmented_control(label, options, default=options[0], key=key) or options[0]
    return st.radio(label, options, horizontal=True, key=key)


def _f(v, dec=1):
    return "—" if v is None or (isinstance(v, float) and pd.isna(v)) else f"{v:.{dec}f}"


def _recompute_score(df, w_temp, w_rain, w_wind):
    total = w_temp + w_rain + w_wind
    w_temp, w_rain, w_wind = w_temp / total, w_rain / total, w_wind / total
    return (
        w_temp * df["temp_comfort_score"].fillna(50)
        + w_rain * df["rain_score"].fillna(50)
        + w_wind * df["wind_score"].fillna(50)
    ).round(1)
```

Map, Forecast, Comparison & City detail

The remaining pages follow the same pattern — load a cached mart, offer a themed filter bar, then draw a Plotly or Folium visual. For example, the Map page maps each variable to a colour ramp and scales each city's marker by its value:

```
VARIABLES = {
    "Visit score": ("visit_score", 0, 100, "score", 0),
    "Temperature": ("fcst_temperature_2m_max", 0, 45, "temp", 0),
    "Precipitation": ("fcst_precipitation_sum", 0, 20, "precip", 1),
    "Wind": ("fcst_wind_speed_10m_max", 0, 80, "wind", 0),
}

# ramp key -> (low rgb, high rgb) using the app's semantic palette
_RAMPS = {
    "score": ((226, 59, 74), (0, 168, 126)), # red -> teal
    "temp": ((55, 138, 221), (226, 59, 74)), # blue -> red
    "precip": ((190, 215, 240), (24, 95, 165)), # pale -> blue
    "wind": ((203, 201, 247), (73, 79, 223)), # pale -> cobalt
}
```

The Forecast page caps the chart at four cities for legibility and colours each city distinctly; the Comparison page normalises every metric to a 0–100 scale so different units can share one radar chart; and the City detail page overlays observed history against the forecast, split at a “Today” marker. In every case the heavy analytical work was already done in the marts — the pages only filter, arrange and draw.

15. Automation — GitHub Actions

The project keeps itself up to date automatically using **GitHub Actions**, a service that runs commands in the cloud on a schedule or when code changes. There are two workflows.

open-meteo-raw-load.yml — the daily pipeline

This runs the entire pipeline end to end — extract, load, then `dbt deps`, `dbt debug`, `dbt parse` and `dbt build`. It fires every day at 06:20 UTC, and can also be started by hand. The secret token is injected securely from the repository's secrets, never stored in the code.

```
name: Open-Meteo raw load

on:
  workflow_dispatch:
  schedule:
    - cron: "20 6 * * *"
  push:
    branches:
      - open-meteo-extraction-load
    paths:
      - ".github/workflows/open-meteo-raw-load.yml"
      - "scripts/extract_open_meteo.py"
      - "scripts/load_raw_to_motherduck.py"
      - "scripts/bootstrap_motherduck.py"
      - "sql/init_motherduck.sql"
      - "requirements.txt"
      - "dbt_project.yml"
      - "profiles.yml"
      - "models/**"
      - "macros/**"

jobs:
  extract-and-load:
    runs-on: ubuntu-latest
    env:
      MOTHERDUCK_DATABASE: open_meteo_europe_sa
      MOTHERDUCK_TOKEN: ${{ secrets.MOTHERDUCK_TOKEN }}

# ... steps: checkout, install Python, check the secret,
# extract 12 cities, load to MotherDuck, then run dbt build ...
```

motherduck-smoke-test.yml — the quick health check

A much lighter workflow that just installs the tools and checks that GitHub can connect to MotherDuck and that the dbt project parses. It is a fast way to confirm the plumbing works without rebuilding everything.

16. Configuration & housekeeping files

A handful of small files keep the project runnable, secure and consistent.

File	Purpose
`.env.example`	A template listing the two secret settings the project needs (the MotherDuck token and database name). Each person copies it to a private `.env` file — which is never committed — and fills in their own token.
`.streamlit/config.toml`	Sets Streamlit's built-in widgets to the project's cobalt brand colour (#494fdf) so native controls match the custom theme.
`requirements.txt`	The list of Python libraries needed to run the pipeline and dashboard (duckdb, dbt-duckdb, streamlit, plotly, folium, pandas, numpy, ...).
`pyproject.toml`	An alternative, modern dependency definition (pins dbt-core 1.8.0) used by the uv package manager and for local development tools like sqlfluff.
`README.md`	The human-readable guide: setup steps, how to run the pipeline, dbt and the dashboard, and how the CI works.
`AGENT.md`	The dashboard design rulebook: the colour tokens, fonts, layout rules and hard constraints (no emojis, one accent colour, preview before committing).
`analyses/sanbox.sql`	A scratch query used during development. Files in `analyses/` are compiled by dbt but never built into the database.
`seeds/ snapshots/ tests/ .gitkeep`	Empty placeholder files that keep standard dbt folders present in version control even when unused.
.gitignore	Lists files Git should ignore — most importantly the secret `.env` file and the local dbt build output.

For reference, the two configuration files small enough to show in full:

.env.example

```
MOTHERDUCK_TOKEN=replace_with_your_motherduck_token
MOTHERDUCK_DATABASE=open_meteo_europe_sa
```

.streamlit/config.toml

```
[theme]
primaryColor = "#494fdf"
font = "sans serif"
```

17. How to run it yourself (quick reference)

The whole project can be reproduced with a short sequence of commands. This is the practical summary a professor could follow.

1. Install and configure

```
python -m venv .venv
.venv\Scripts\Activate.ps1 # Windows PowerShell
python -m pip install -r requirements.txt

copy .env.example .env # then paste your MotherDuck token
```

2. Create the database objects (once)

```
python scripts/bootstrap_motherduck.py
```

3. Extract and load the raw data

```
python scripts/extract_open_meteo.py --output-dir data/raw --past-days 92 --forecast-days 7
python scripts/load_raw_to_motherduck.py --input-dir data/raw
```

4. Build and test all the dbt models

```
dbt deps --profiles-dir . # install dbt_utils
dbt debug --profiles-dir . # check the connection
dbt build --profiles-dir . # run + test every model
```

5. Launch the dashboard

```
streamlit run dashboard/app.py
```

In one sentence: the scripts fill the `raw` layer, `dbt build` transforms it through `staging → intermediate → marts` while testing every step, and Streamlit reads the finished marts to power the six-tab dashboard.

18. Glossary

Plain-English definitions of the technical terms used in this document and the project.

Term	Meaning
API	“Application Programming Interface”. A way for one program to request data from another over the internet — here, Open-Meteo's weather service.
Analytics engineering	The discipline of turning raw data into clean, reliable, well-tested datasets ready for analysis — the main theme of this project.
Warehouse	A database designed for analysis over large amounts of data. This project uses MotherDuck.
Schema	A named folder inside a database that groups related tables. Here: raw, staging, intermediate, marts.
dbt	“data build tool”. Software that lets you define data transformations as SQL files, with automatic ordering, testing and documentation.
Model	A single dbt transformation — one `.sql` file that produces one table or view.
View	A saved query that runs fresh every time it is read. Takes no storage and is always current. Staging models are views.
Table (materialized)	A query whose results are computed once and stored on disk for fast reading. Intermediate and marts are tables.
Materialization	dbt's term for whether a model is built as a view or a stored table.
Source	A raw input table that dbt reads but does not build, declared in `_sources.yml`.
ref() / source()	dbt functions used to reference another model or a source. They let dbt work out the build order automatically.
Grain	The level of detail of one row in a table, e.g. “one row per city per day”.
Surrogate key	A single made-up identifier for a row (e.g. city id + date joined by a dash) used to guarantee uniqueness.
Dimension	A table of descriptive attributes (e.g. city name, country) that facts point to. Here: `dim_location`.
Fact table	A table of measurements/events (e.g. a day's weather), usually the largest tables. Prefixed `fct_`.
Mart	A final, purpose-built dataset ready for a specific use (here, the dashboard).
Deduplication	Removing repeated copies of the same record, keeping only the most recent.
Window function	A SQL feature that ranks or numbers rows within groups — used here to find the newest copy of each record.
Upsert	“Update or insert”: refresh existing rows and add new ones, avoiding duplicates when a load runs repeatedly.
Natural key	The real-world columns that uniquely identify a record (e.g. city + date), used to match rows during an upsert.
AQI	“Air Quality Index”. A single number summarising air pollution; classified into bands from good to extremely poor.
Visit score	This project's headline 0–100 rating of a city's holiday weather (40% temperature, 35% low rain, 25% low wind).

Term	Meaning
Forecast horizon	How many days ahead a forecast is looking (forecast date minus the day it was made).
MAE / signed error	Mean Absolute Error averages how far off forecasts were, ignoring direction; signed error keeps direction (too high / too low).
Environment variable	A setting stored outside the code (like a secret token), so credentials never appear in the source files.
CI/CD	Automation that runs tasks (tests, builds, the pipeline) automatically. Here provided by GitHub Actions.
Streamlit	A Python tool for building interactive web dashboards without writing HTML or JavaScript by hand.

End of document. Generated automatically from the project's own source files, so every code excerpt matches the repository exactly.